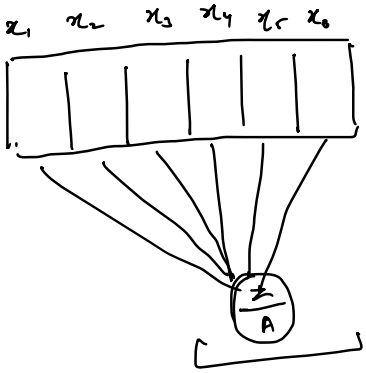
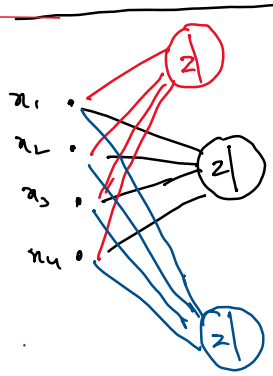
Neural Network

06 August 2026 09:41



Basic logistic Regression

$w_1 = 0.3$
$b_1 = 0.1$
w_2 w_3.



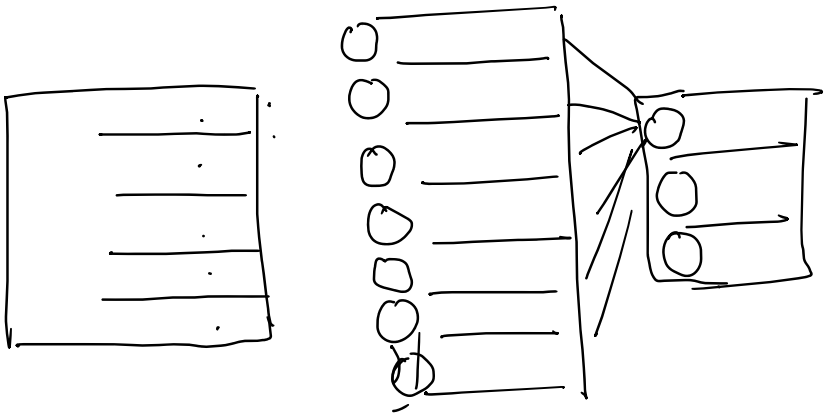
$w_1 = 0.1$
$b_1 = 0.2$
0

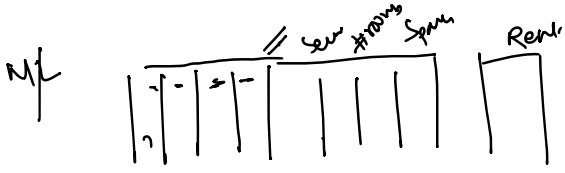
$w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$

$w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b_{in}$

$w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b_{in}$

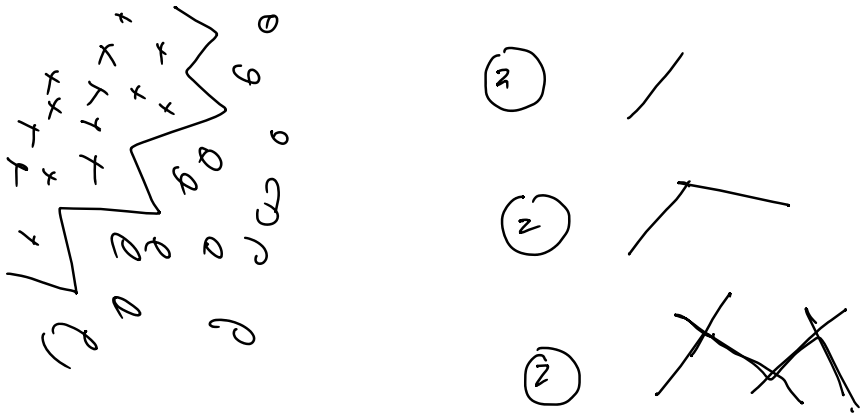
randn $[-0.1, -0.2, 0, +0.3, +0.42]$

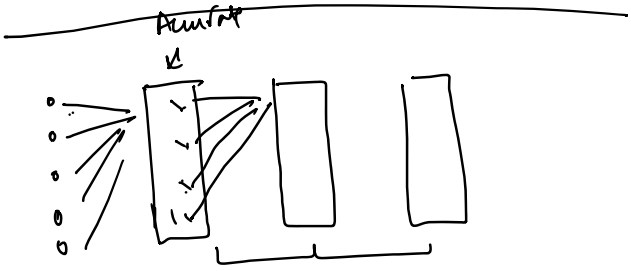


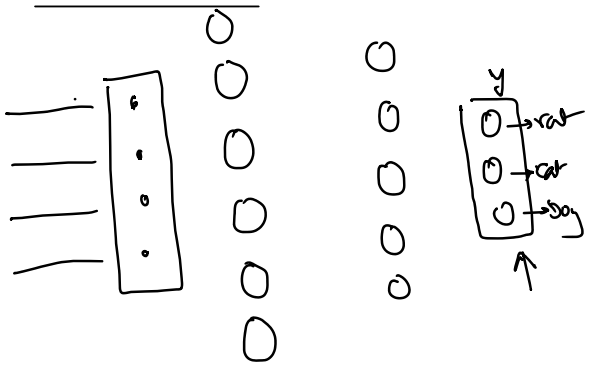


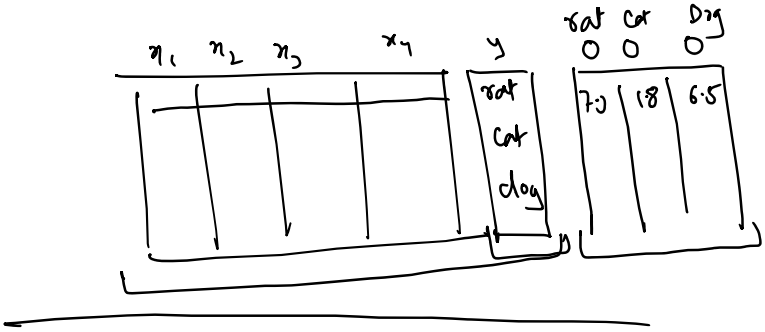
C >> -
C ~ R

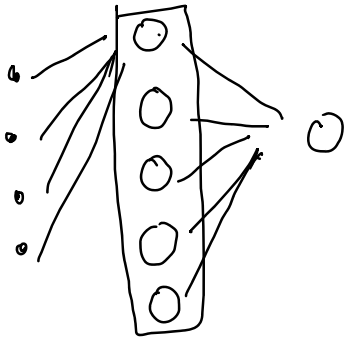
Extra knowlge leads to overfitting

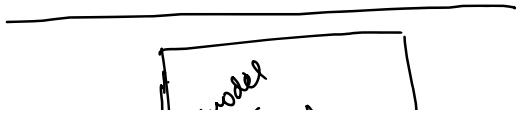


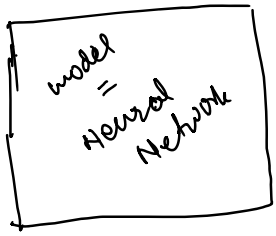



y
rat
cat
dog


x1 x2 x3 x4
y
rat
cat
dog
rat cat Dog
7.0 1.8 6.5

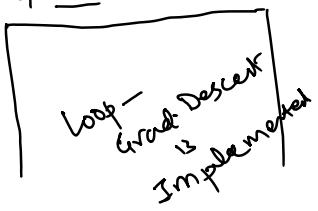


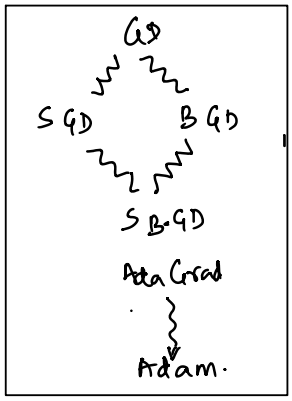




Loss function

Optimizer function

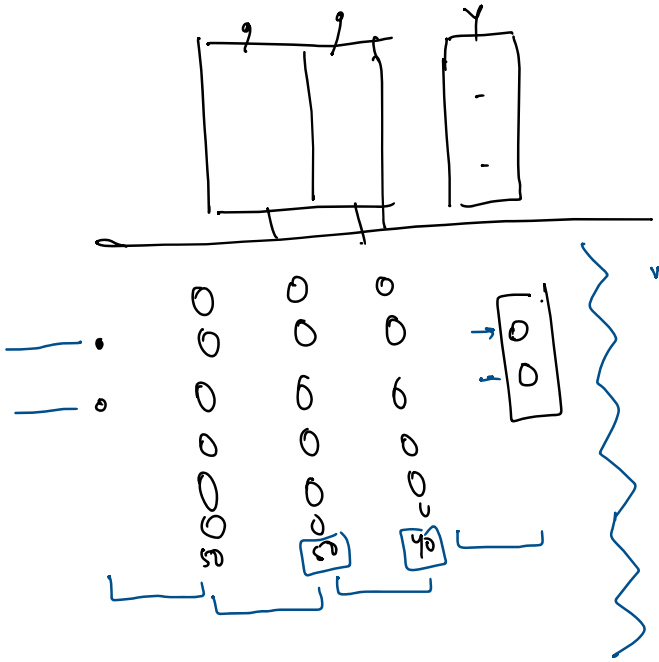




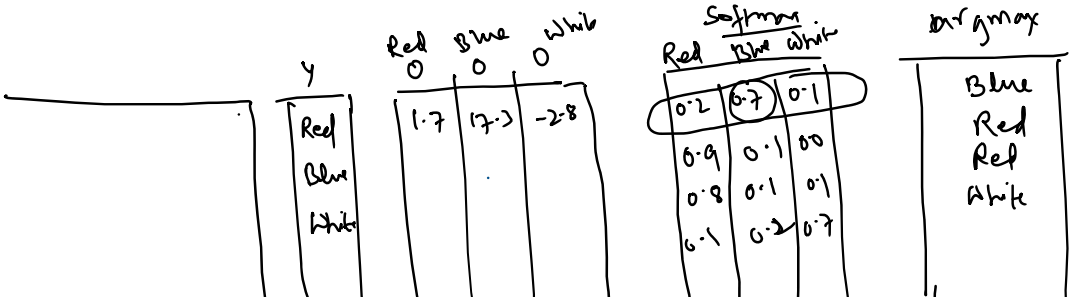
$$GD \Rightarrow \quad w = w - \alpha * w.grad$$

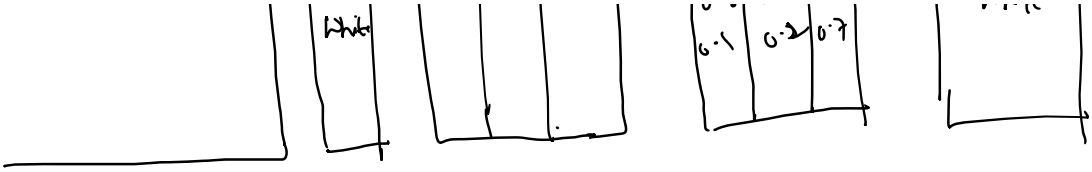
$$w = w - \alpha \left(\frac{d\,Loss}{dw} \right)$$

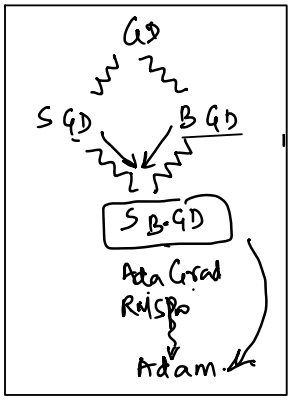
$$Loss = Avg\ e^2$$



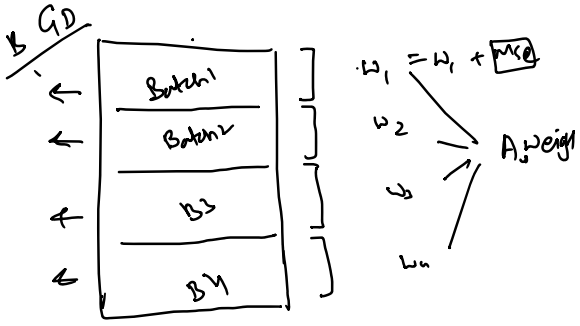

```
model = Sequential(
    nn.Linear(2, 50)
    nn.Linear(50, 50)
    nn.Linear(50, 40)
    nn.Linear(40, 2)
    )
```





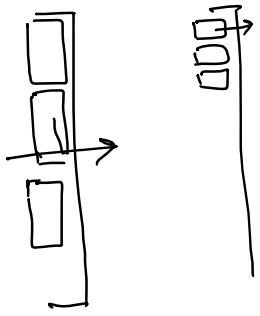


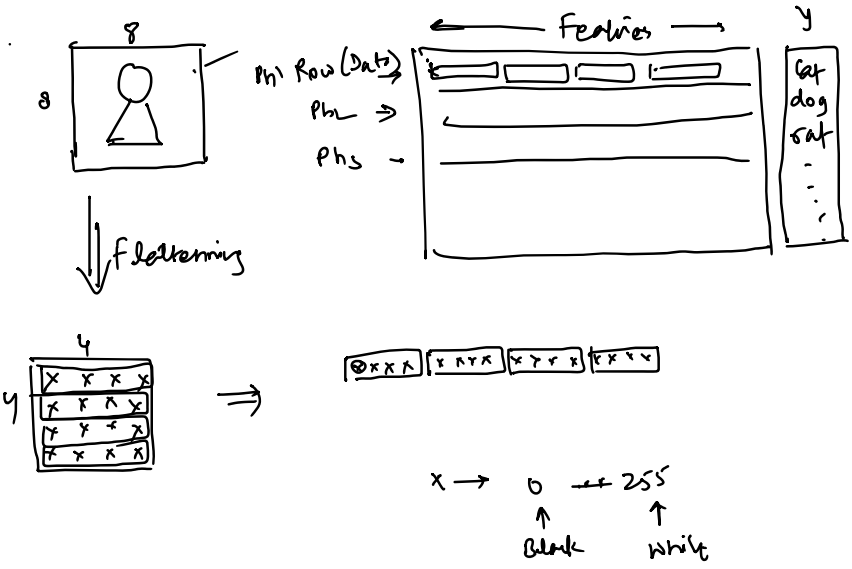
Loss → Avg e^2

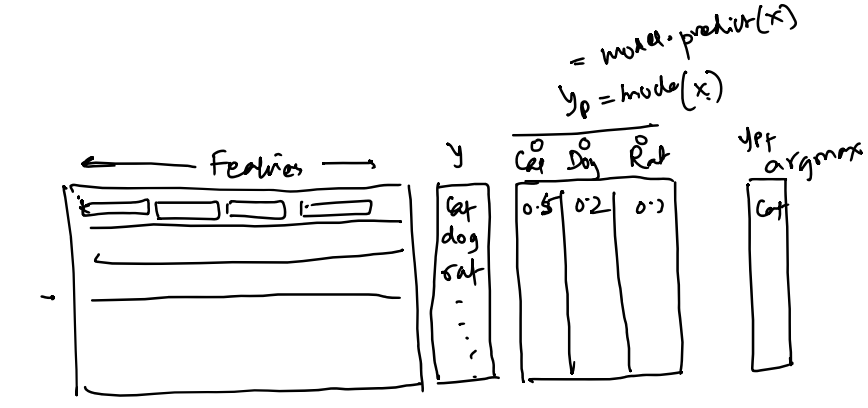


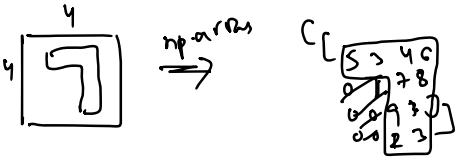
M.B GD → (Mini)

SGD [Stochastic Gradient Descent]









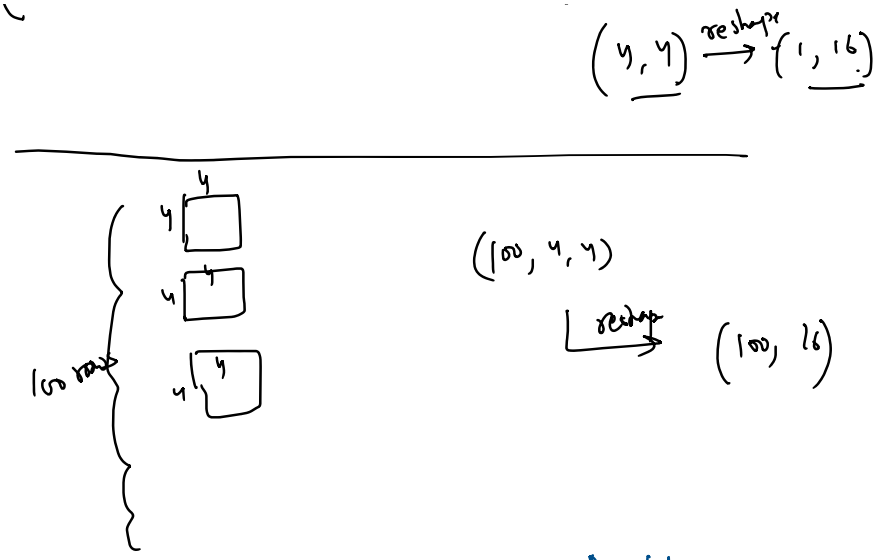
not required in sklearn

```
!pip install opencv-python

import cv

cv.imread(Image) → np-array.
```

(4, 4) reshape (1, 16)

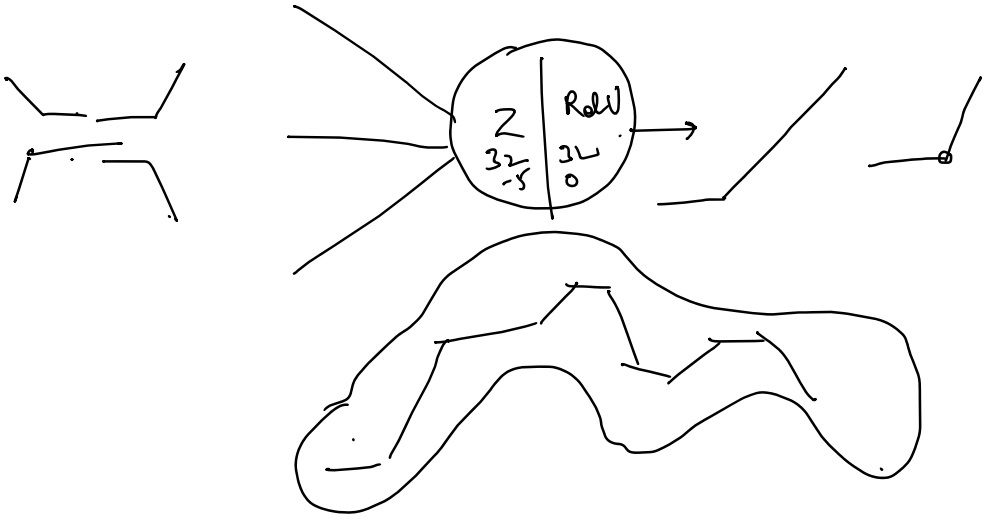


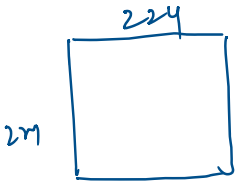
Data set
Hand Written
Images

```
array([[ 0.,  1., 10., 13.,  2.,  0.,  0.],
       [ 0., 10., 16., 16., 12.,  0.,  0.],
       [ 0.,  9.,  9.,  8., 16.,  0.,  0.],
       [ 0.,  0.,  0.,  6., 16.,  2.,  0.],
       [ 0.,  0.,  1., 11., 15.,  0.,  0.],
       [ 0.,  0.,  4., 16., 13.,  2.,  0.],
       [ 0.,  0., 14., 16., 16., 13.,  0.],
       [ 0.,  0.,  9., 13., 11., 10.,  9.,  0.]])
```

Images	labels
0	0
1	1

0 1 2





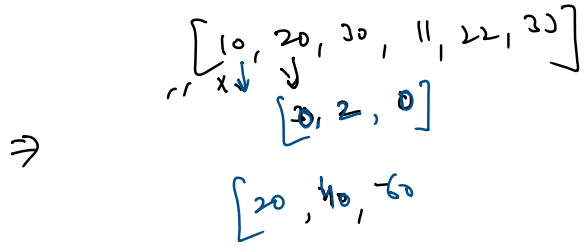
— 224 × 224 × 3 —

Convolution

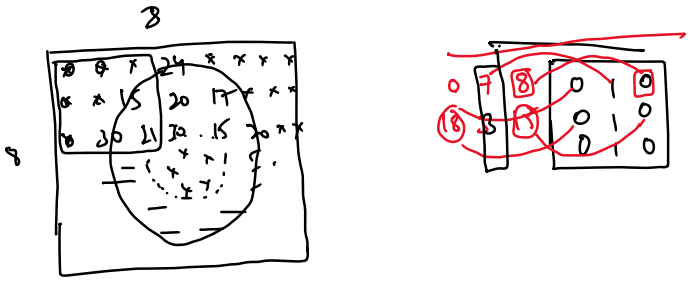
```
ar = np.array([10,20,30,11,22,33])
```

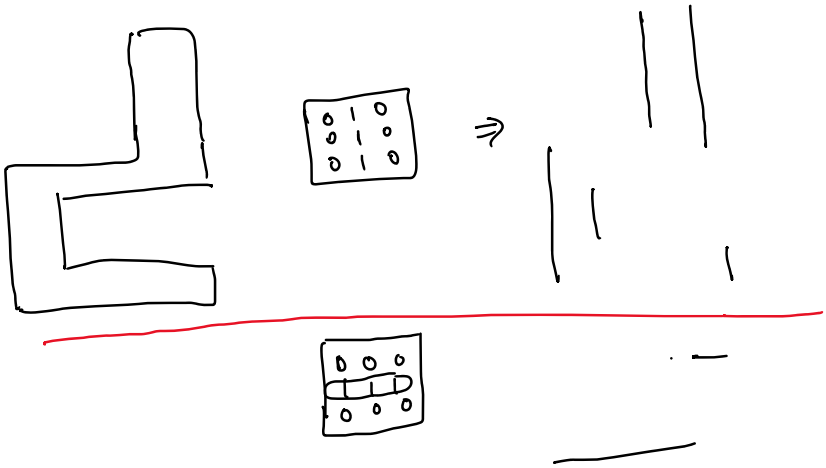
```
np.convolve(ar, [1,2,3]) # [3,2,1]
```

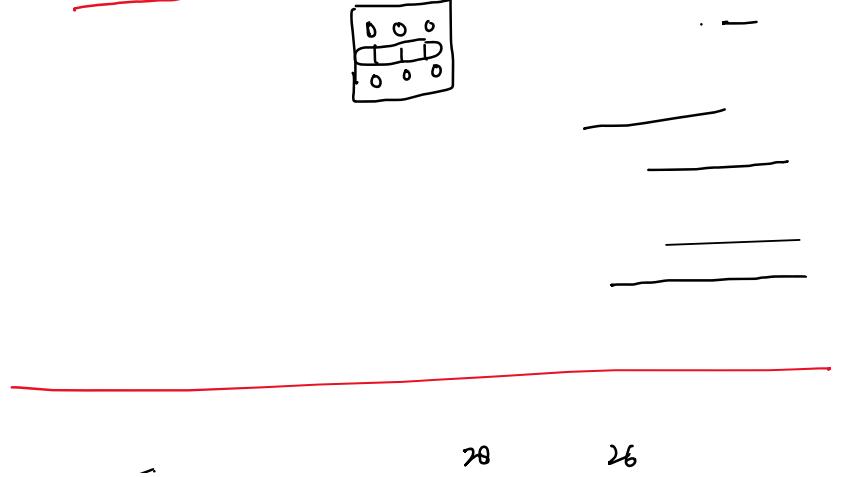
```
array([ 10,  40, 100, 131, 134, 110, 132,  99])
```



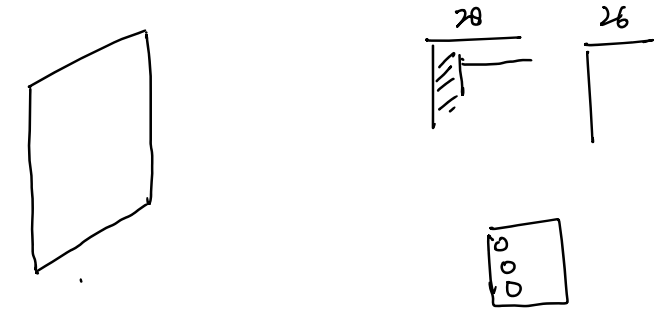
⇒ [20, 40, 60

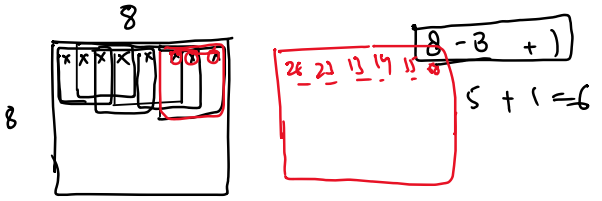


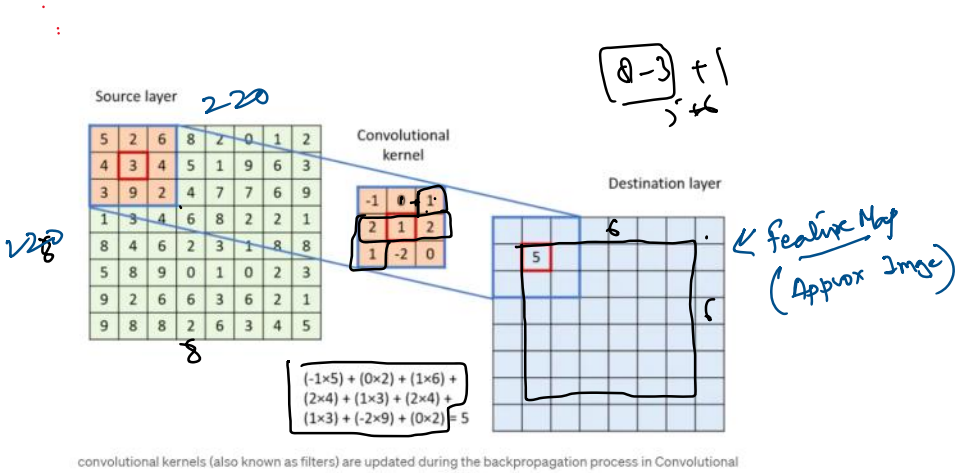




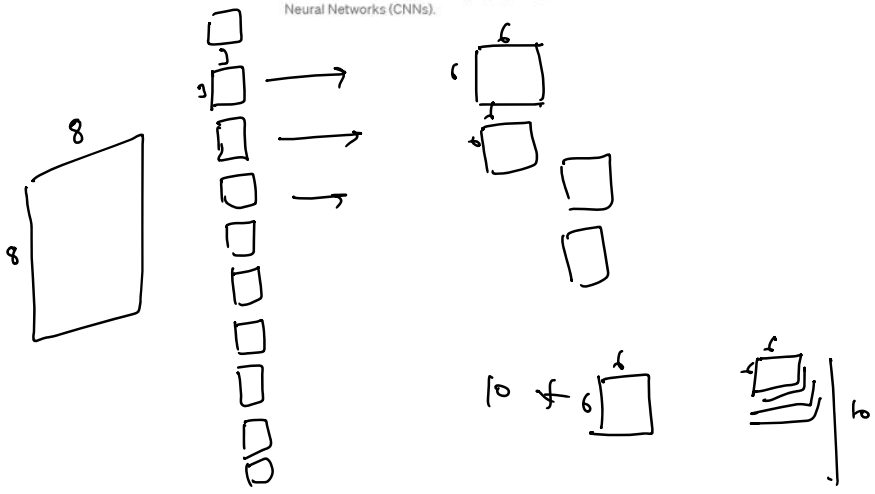
Quick Notes Page 7

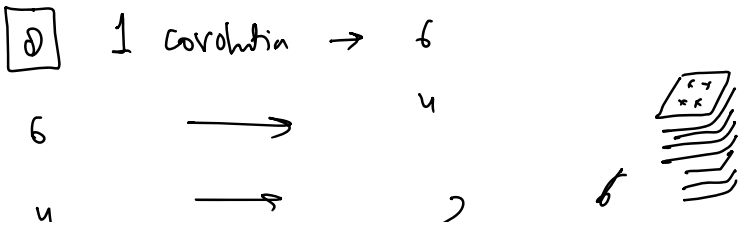


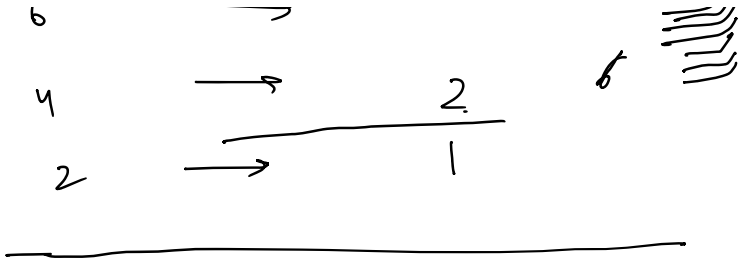


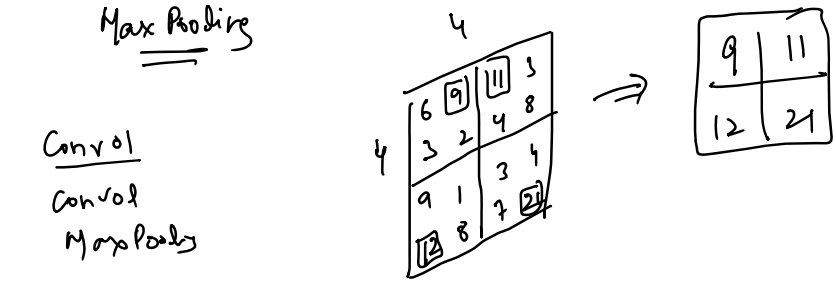


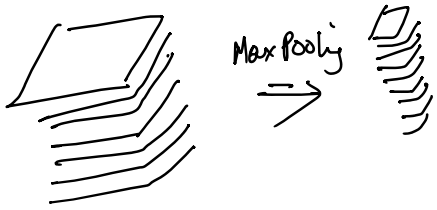
convolutional kernels (also known as filters) are updated during the backpropagation process in Convolutional Neural Networks (CNNs).

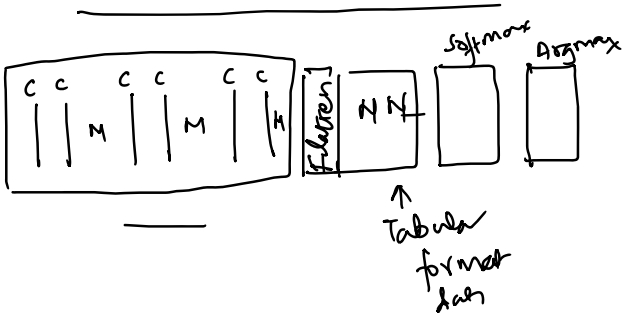





6
4
2
2
1
6


Max Pooling
Convol
Convol
MaxPooly
4
4
6 9 11 3
3 2 4 8
9 1 3 4
12 8 7 21
9 11
12 21


MaxPooly


C C C C C C
M M M
F
Flatten
N N
Softmax
Argmax
Tabular
format
data